

PLATFORM ENGINEERING CUSTOMER PITCH

KAOP

by Nirit Terehovsky

A governed operating layer for production agents

Customer presentation and live platform tour



The agent sprawl problem

Independent adoption creates an operating problem for the platform team



App team

Coding agent



SRE

Incident agent



FinOps

Cost agent



Security

Policy agent

PLATFORM QUESTIONS

What is running across the organization?

Which systems and credentials can each agent reach?

Who approved the action and what evidence remains?

How do we stop unsafe behavior without stopping adoption?

Why isolated agents fail in production

A useful local agent still needs production controls around it

SCOPED ACCESS

Who can read or write, in which environment

RUN EVIDENCE

Inputs, tool calls, decisions, outcomes, and cost

Local agent

RELIABLE TRIGGERS

Alerts, schedules, pull requests, or chat

APPROVAL GATES

Human review before higher risk actions

The KAOP control plane

Shared infrastructure surrounds both Komodor agents and custom agents

Fleet

Inventory, ownership, health, versions

Run

Triggers, workflows, agent handoffs, run history

Context

Memory, knowledge graph, integrations

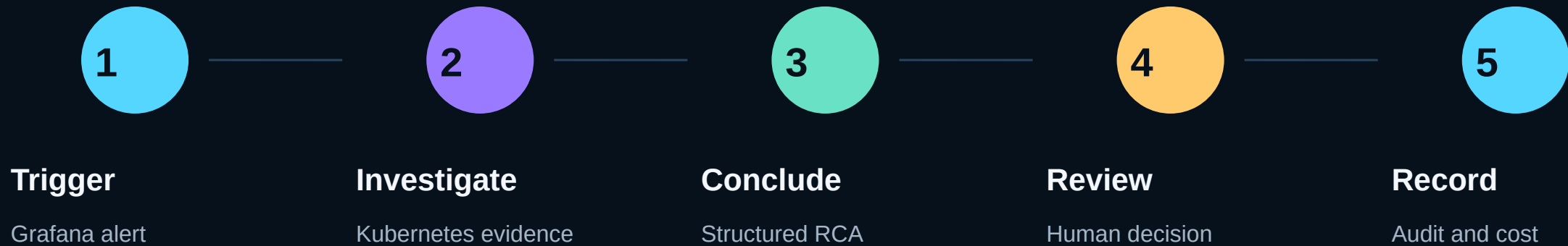
Govern

RBAC, credentials, guardrails, approvals, audit

One operating model across runtimes, models, and teams

Workflow orchestration from trigger to outcome

A defined workflow turns agent capability into a repeatable operational process



Output contract

root cause / evidence / confidence / affected resources / suggested fix

Governance boundaries

The demo proves control through enforced permissions, not through instructions alone

Kubernetes investigator

ALLOWED

- + Pods and logs
- + Events and workloads
- + Services and endpoints

DENIED

- Secrets
- Cluster scope
- Any write

GitHub reviewer

ALLOWED

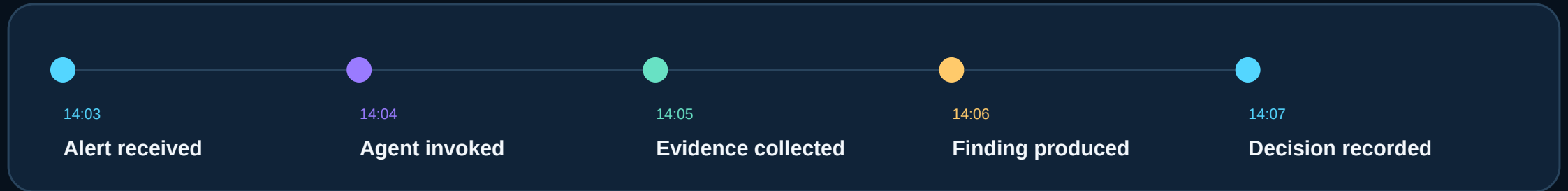
- + Repository content
- + Pull request diff
- + Review comments

DENIED

- Push
- Merge
- Code changes

Fleet visibility and auditability

Central evidence replaces reconstruction from local terminals and team-specific logs



OWNERSHIP

Who owns this agent and workflow?

ACCESS

What could the agent read or change?

EVIDENCE

Which inputs and tool calls support the result?

ECONOMICS

How much did the run consume?

The run record makes every action attributable

Live platform tour

The demo starts with controls, then follows live evidence across three systems

KAOP

Verified RCA run
High confidence finding

Show the boundary before the run

GRAFANA

Normal / Pending / Firing /
Normal

Prove the trigger was automatic

GITHUB

Open PR #1 with AgentOps
review

Prove the reviewer stayed comment only

Live trigger

make incident

Redis endpoints disappear, probes fail, and
restarts rise

A governed path to agent adoption

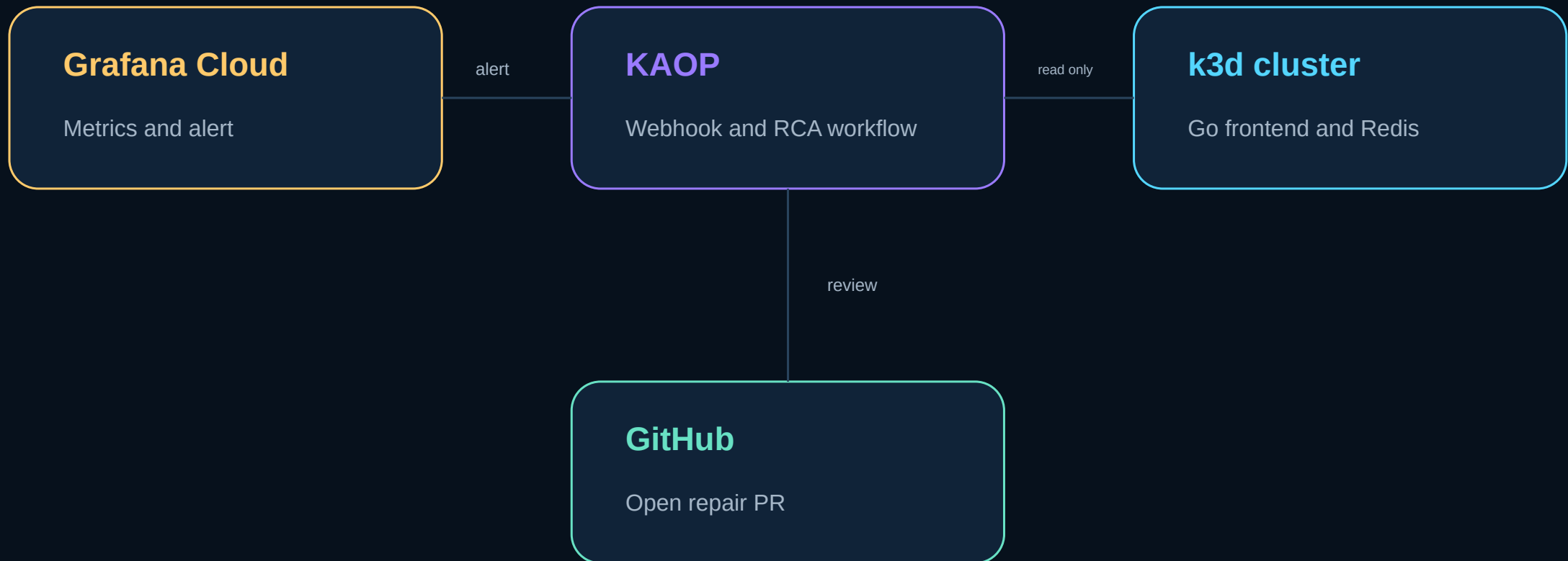
Autonomy expands only after the organization has evidence that the workflow is reliable



Start where the blast radius is clear and the outcome is measurable

Demo architecture

One application and one fault connect the incident and code review workflows



Fault: Redis Service selector matches no Pods

Permission matrix

The demo uses the minimum access required for investigation and review

Integration	Capability	Allowed	Enforcement
Kubernetes	Pods, logs, events	Read	Namespace Role
Kubernetes	Services, endpoints, workloads	Read	Namespace Role
Kubernetes	Secrets and writes	Denied	RBAC
GitHub	Repository and PR diff	Read	App permission
GitHub	Review comments only	Write comment	App permission

No credentials or Secret values are stored in Git

Incident evidence and recovery

Kubernetes, Grafana, and KAOP expose the fault and the recovery

Healthy

1

Redis endpoint

2/2 Ready

0 restarts

Incident

0

Redis endpoints

new Pod not Ready

7 restarts observed

Recovered

1

Redis endpoint

2/2 Ready

alert resolved

Recovery command

make recover && make verify